Does a Compressed Spectral Loss Help DCCRN Enhance Speech?

Bharath Srividhya, Jason Redick, Ioannis Kontothanassis
bksri27@bu.edu, jredick@bu.edu, alexiosk@bu.edu

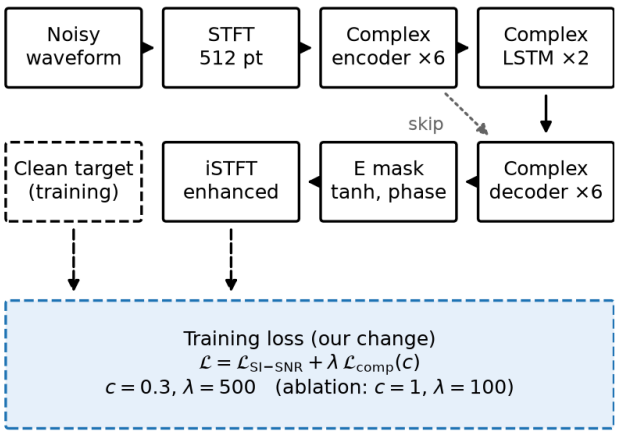


Figure 1. DCCRN pipeline. The network and data stay the same as the baseline. Our one change is the training loss on the output (blue box).

1. Task and Hypothesis

This project focuses on single channel speech enhancement. The model takes a noisy 16 kHz audio recording and outputs an estimated clean version of the same length. Our baseline is DCCRN-E [1], a complex convolutional recurrent network trained with SI-SNR loss. While SI-SNR improves waveform accuracy, it ignores the finer spectral details that perceptual metrics like PESQ measure.

Our only change is the loss function. We add a power law compressed spectral loss [2] to the SI-SNR loss. Compressing magnitudes with an exponent $c < 1$ gives quiet time frequency bins more weight than a standard MSE loss would. We hypothesize that this will improve PESQ and STOI scores without hurting SI-SDR. We tested $c = 0.3$, alongside a $c = 1$ ablation run. The $c = 1$ run adds the spectral term without any compression, allowing us to isolate the specific impact of the compression itself.

2. Method

Figure 1 illustrates our pipeline. We use the original DCCRN-E configuration and the authors' reference code. The STFT uses a 400 sample square root Hann window, a 100 sample hop size, and a 512 point FFT. The causal model has 3.67M parameters. It consists of six complex convolutional layers (32, 64, 128, 256, 256, and 256 channels) that feed into a two layer complex LSTM and a mirrored complex decoder with skip connections. The final mask limits the magnitude using tanh and applies a phase rotation to the noisy input.

The proposed loss is $L = L_{\text{SI-SNR}} + \lambda L_{\text{comp}}(c)$. The compression term L_{comp} combines a compressed magnitude MSE (0.7 weight) and a compressed complex MSE (0.3 weight) [2]. We set $\lambda = 500$ for $c = 0.3$ and $\lambda = 100$ for $c = 1$. This ensures the spectral term has roughly the same initial gradient norm as the SI-SNR loss.

All three systems share the same training recipe. We use the Adam optimizer with a 1e-3 learning rate, a batch size of 16, aligned 4 second random crops, and gradient clipping at 5. If the dev loss does not improve for two epochs, the learning rate halves. Training stops early after 10 epochs without improvement in dev A(D), and we keep the model from the best dev epoch.

3. Data and Metrics

We use the 16 kHz VoiceBank-DEMAND dataset [3], which contains 11,572 noisy and clean audio pairs. We reserve two speakers for the dev set: p226 (male) and p230 (female). This splits the data into 10,827 training files and 745 dev files. The dev set features 10 noise types at 0, 5, 10, and 15 dB SNR. Since these exact noise types and SNR levels also exist in the training set, the dev set only measures performance on unseen speakers.

We track PESQ, STOI, and SI-SDR, as well as a composite score called A(D). A(D) is a weighted average of normalized PESQ (0.4), STOI (0.3), and SI-SDR (0.3). The final dev score is out of 100, calculated as $100 \times (0.85\,A(\text{noisy}) + 0.15\,A(\text{clean input}))$, following the official leaderboard formula. All scores are generated using the official enhance.py and evaluate.py scripts.

4. Baseline and Initial Results

Table 1 gives dev results. Each system is one training seed.

System	PESQ	STOI	SI-SDR	A noisy	A clean	Score
Noisy input	1.554	0.826	6.27	0.433		
DCCRN (SI-SNR)	2.414	0.872	15.59	0.615	0.920	66.09
+ comp, c = 0.3	**2.681**	**0.884**	15.57	**0.649**	**0.931**	**69.12**
+ comp, c = 1	2.423	0.875	15.56	0.617	0.927	66.33

Table 1. Dev results, 745 utterances, one seed each. SI-SDR in dB. "A clean" scores the model run on clean input.

We compared each system to the DCCRN baseline using a paired bootstrap test over the 745 dev files (10,000

resamples, 95% confidence intervals). The $c = 0.3$ model improves the overall dev score by +3.03 [+2.88, +3.19] and beats the baseline on 92% of the files. PESQ increases by +0.266 [+0.252, +0.281] and STOI by +0.0116 [+0.0103, +0.0129], while the change in SI-SDR is not significant: −0.02 [−0.05, +0.01] dB.

The $c = 1$ ablation only improves the score by +0.25 [+0.15, +0.34], outperforming the baseline on just 55% of files. This suggests that the performance boost comes from the compression, not just from adding a spectral loss. Note: these intervals only account for utterance sampling variance, not variation between training seeds.

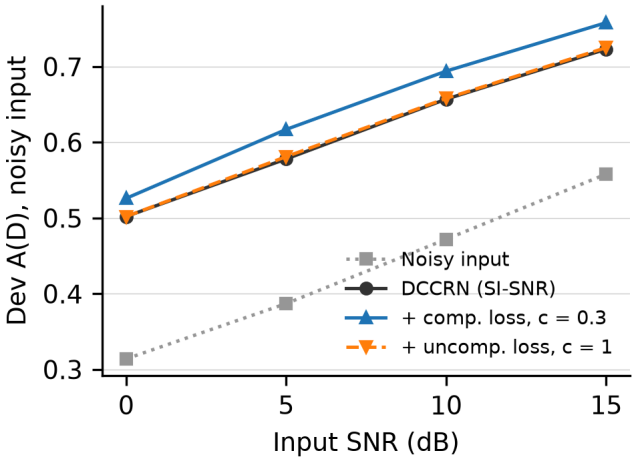


Figure 2. Dev A(D) on noisy input by input SNR.

Figure 2 shows that $c = 0.3$ outperforms the baseline across all SNR levels. The improvement is smallest at the hardest 0 dB level (+0.024 [+0.021, +0.028] in A(D)), compared to gains of +0.034 to +0.038 at the higher SNRs.

Figure 3 shows that $c = 0.3$ improves performance across all 10 noise types. In contrast, the $c = 1$ ablation has mixed results, and performs significantly worse than the baseline on car noise (−0.0077 [−0.0113, −0.0044]).

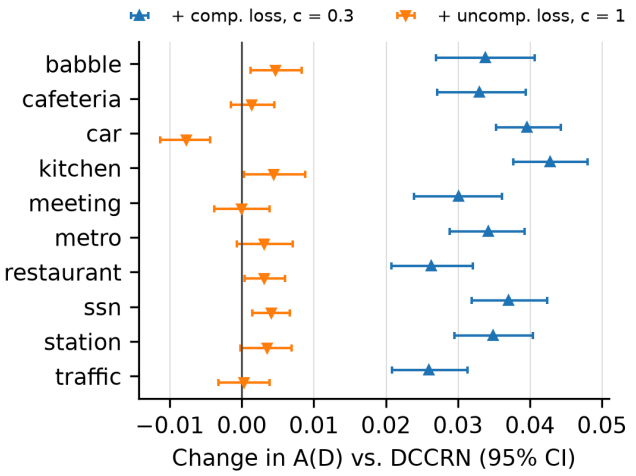


Figure 3. Paired change in dev A(D) against DCCRN by noise type, with 95% bootstrap intervals.

5. Metric Checks

- **Data validation:** We confirmed all 11,572 files have the correct sample rate, format, length, and SNR matching the corpus log. Cross correlation shows zero alignment lag for all pairs.
- **Scoring accuracy:** The official evaluation scripts exactly match the scores output by our training loop.
- **Output range:** On the dev set, no output peak goes above 1.0 and no PESQ computation fails.
- **Clean input test:** Passing clean speech through the models yields an A(D) score of 0.920 for DCCRN and 0.931 for $c = 0.3$.
- **Edge cases:** We tested the submitted model on 22 unusual inputs, including silence, extremely quiet audio, a 2 minute file, tiny files of 1 to 1,000 samples, and maximum volume signals. The model kept the original length and produced finite, identical outputs on repeated runs every time. However, three full volume inputs (a heavily clipped speech like signal, a 1 kHz sine wave, and a 200 Hz square wave) produced output peaks above 1.0. The official runner would reject these specific outputs.

6. Resource Use

Item	Value
Parameters	3,671,053 (limit 15M)
Checkpoint size	14.7 MB
Peak training memory	6.6 GiB on one L40S (limit 32 GiB)
Training time	2.5 to 2.8 GPU hours per run
Real time factor	about 0.006 on L40S, 0.10 on an Apple M2 CPU

7. Risks

- A disk quota crash interrupted the DCCRN baseline, so it may have trained for fewer epochs than the new models.
- We only ran one seed per system, so we cannot rule out random seed variation yet.
- Because we kept the best epoch based on the dev set, the dev scores might be slightly optimistic.
- The model improves the least at 0 dB SNR, which is the most challenging noise condition.
- The model lacks an output limiter. As noted in Section 5, unusually loud or clipped inputs can push the output above the strict 1.0 peak limit, and the runner rules do not permit clipping.

8. Planned Ablations and Next Steps

1. Train a second seed for both the baseline and the $c = 0.3$ model to measure seed variance.
2. Test different λ values around 500 to ensure the improvements are not just tied to a single lucky weight.

3. Listen to and plot spectrograms for the most extreme successes and failures, focusing closely on files where PESQ and SI-SDR disagree.
4. Identify the exact input volume that causes the model to output peaks above 1.0, and figure out a way to fix it within the rules.
5. If time permits, test $c = 0.5$ to see how changing the compression exponent affects the final gain.

9. Contributions

- **Ioannis Kontothanassis:** DCCRN model, training script and the compressed loss, most training runs, the dev comparison script and the submitted checkpoint.
- **Jason Redick:** SCC batch job scripts, checkpoint resume fix, and dev evaluation through the official chain.
- **Bharath Srividhya:** helped design and debug the model and training code in shared sessions, wrote the figure script (report/make_figures.py) and the edge case test (cli/edge_cases.py), ran the edge case checks, and wrote this report.

AI use. We used Claude, an AI assistant, in three ways. It helped write two support scripts (report/make_figures.py and cli/edge_cases.py), which we reviewed and ran ourselves. It produced a first draft of this report from our results and notes, which we then rewrote in our own words and checked against our logs. It also checked grammar. All experiments, results and design decisions are our own.

References

1. Y. Hu, Y. Liu, S. Lv, M. Xing, S. Zhang, Y. Fu, J. Wu, B. Zhang, L. Xie. DCCRN: Deep Complex Convolution Recurrent Network for Phase Aware Speech Enhancement. Interspeech, 2020.
2. S. Braun, I. Tashev. A Consolidated View of Loss Functions for Supervised Deep Learning Based Speech Enhancement. TSP, 2021.
3. C. Valentini-Botinhao, X. Wang, S. Takaki, J. Yamagishi. Investigating RNN Based Speech Enhancement Methods for Noise Robust Text to Speech. SSW, 2016.